

Question-01:- prove Fermat's theorem from group theoretic first principles, then explain how it motivates the Miller-Rabin probabilistic primality test. Apply the test (base $a=2$) to $n=561$ and explain why 561 is a Carmichael number that fools the simpler Fermat test.

⇒ FLT - group theoretic proof:

$(\mathbb{Z}/p\mathbb{Z})$ is a group of order $p-1$ (every non zero element mod prime p has an inverse). For $p \nmid a$, $\bar{a}$ is an element of this group. By Lagrange's theorem, $\text{ord}(\bar{a}) \mid (p-1)$, so $p-1 = k \cdot \text{ord}(\bar{a})$ and

$$a^{p-1} = (a^{\text{ord}(\bar{a})})^k \equiv 1^k = 1 \pmod{p}$$

Motivation for Miller-Rabin :-

$\mathbb{Z}/p\mathbb{Z}$ is a field when p prime, so $x^2 \equiv 1$ has only roots ± 1 . Write $n-1 = 2^s d$ (d odd) and repeatedly square a^d . If n is prime, the chain can only hit 1 via a preceding -1. If it hits 1 from something other than ± 1 , that's a fake square root of unity - impossible mod a prime.

so n must be composite.

Test on $n = 561$ $a = 2$.

$$560 = 2^4 \cdot 35, \text{ so } s=4, d=35.$$

$$2^{35} = 263, \quad 2^{70} = 166, \quad 2^{140} = 67, \quad 2^{180} = 1 \pmod{561}$$

Since $67^4 = 1$ but $67 \not\equiv \pm 1 \pmod{561}$, this is a nontrivial root of unity. 561 is composite.

Why 561 is Carmichael:

$561 = 3 \cdot 11 \cdot 17$ (squarefree). Korselt's criterion: $(p-1) \mid (n-1)$ for each prime factor;

$$2 \mid 560, \quad 10 \mid 560, \quad 16 \mid 560.$$

All hold, so 561 is Carmichael. By CRT this forces $a^{560} \equiv 1 \pmod{561}$ for every a coprime to 561 . Then plain Fermat test can never catch it, regardless of base.

Question-2:- prove that $\mathbb{Z}_n$ under addition and multiplication modulo n is a field if and only if n is prime. Illustrate your proof by contrasting the multiplicative structure of $\mathbb{Z}_{11}$ with $\mathbb{Z}_{12}$.

⇒ Theorem $\mathbb{Z}_n = (\mathbb{Z}/n\mathbb{Z}, +, \cdot)$ is a field iff n is prime

($\mathbb{Z}_n$ is always a commutative ring) Addition and multiplication mod n are well defined, associative, commutative, with identities $0, 1$, additive inverse exists, and distributivity holds, so $\mathbb{Z}_n$ is always a commutative ring with 1 . The only extra thing needed for a field is that every nonzero element has a multiplicative inverse. write $n = ab$ with $1 < a, b < n$. Then a, b are nonzero elements of $\mathbb{Z}_n$, but

$$\boxed{a \cdot b = n = 0 \pmod{n}}$$

so a, b are nonzero zero-divisors. If a had a multiplicative inverse a^{-1} , then multiplying $ab \equiv 0$ by a^{-1} gives $b \equiv 0$.

$$ak + py = 1$$

$$\Rightarrow ak \equiv 1 \pmod{p}$$

so $k \pmod{p}$ is a multiplicative inverse of a . Every nonzero element is invertible, so $\mathbb{Z}_p$ is a field.

Illustration: $\mathbb{Z}_{11}$ vs $\mathbb{Z}_{12}$

$\mathbb{Z}_{11}$ (prime) every nonzero element $1, \dots, 10$ is invertible.

$$2 \cdot 6 = 12 \equiv 1, 3 \cdot 4 = 12 \equiv 1, 5 \cdot 9 = 45 \equiv 1$$

$$7 \cdot 8 = 56 \equiv 1 \pmod{11}$$

$\mathbb{Z}_{12}$ (Composite $12 = 3 \cdot 4$): zero divisor appears

$$3 \times 4 = 12 \equiv 0, 2 \times 6 = 12 \equiv 0, 6 \times 6 = 36 \equiv 0 \pmod{12}$$

Elements like 2, 3, 4, 6, 8, 9, 10, have no multiplicative inverse, so not every non zero element is invertible. only elements coprime to 12 - $\{1, 5, 7, 11\}$ are invertible, forming the smaller group $(\mathbb{Z}_{12}, \times)$ of order $\varphi(12) = 4$.

Question 303: - Construct $GF(2^3)$ using the irreducible polynomial $x^3 + x + 1$ over $GF(2)$. Build the multiplication table of a generator element α , and determine whether the multiplicative group $GF(2^3)^*$ is cyclic.

$\Rightarrow GF(2^3)$ here should be understood as $GF(2^3) = GF(8)$.

Given irreducible polynomial:

$$f(x) = x^3 + x + 1$$

Let α be a root of $f(x)$. Then

$$\alpha^3 + \alpha + 1 = 0$$

over $GF(2)$

$$\alpha^3 = \alpha + 1$$

Thus, $GF(8) = \{0, 1, \alpha, \alpha^2, \alpha+1, \alpha^2+\alpha, \alpha^2+\alpha+1, \alpha^2+\alpha^2+\alpha+1\}$

power of α

$$\text{using } \alpha^3 = \alpha + 1$$

$$\alpha^0 = 1$$

$$\alpha^1 = \alpha$$

$$\alpha^2 = \alpha^2$$

$$\alpha^3 = \alpha + 1$$

$$\alpha^4 = \alpha^2 + \alpha$$

$$\alpha^5 = \alpha^2 + \alpha + 1$$

$$\alpha^6 = \alpha^2 + 1$$

$$\alpha^7 = 1$$

Therefore, $\alpha^7 = 1$.

multiplicative table of the generator.

since, $\alpha^i \alpha^j = \alpha^{(i+j) \bmod 7}$.

we get,

x	1	α	α^2	α^3	α^4	α^5	α^6
1	1	α	α^2	α^3	α^4	α^5	α^6
α	α	α^2	α^3	α^4	α^5	α^6	1
α^2	α^2	α^3	α^4	α^5	α^6	1	α
α^3	α^3	α^4	α^5	α^6	1	α	α^2
α^4	α^4	α^5	α^6	1	α	α^2	α^3
α^5	α^5	α^6	1	α	α^2	α^3	α^4
α^6	α^6	1	α	α^2	α^3	α^4	α^5

Is $GF(8)^*$ cyclic?

The multiplicative group contains,

$$|GF(8)^*| = 8-1 = 7 \text{ elements.}$$

since α has order 7,

$$GF(8)^* = \langle \alpha \rangle.$$

Therefore, $GF(8)^*$ is cyclic, with generator α .

Question-04:- Derive the time complexity of modular expansion via square and multiple. Compute 7^{560} and 561 step by step and interpret the result in light of 561 being a Carmichael number.

⇒ square and multiply:-

To calculate $a^e \bmod n$, write e in binary. The algorithm repeatedly squares the current value and multiplies when the binary bit is 1.

If e has $k = \lfloor \log_2 e \rfloor + 1$ bits, there are at most $(k-1)$ squaring and $(k-1)$ multiplications.

$$T(e) = O(\log e).$$

For, $e = 560$

$$560 = (1000110000)_2$$

Now calculate modulo 561 :

$$2^1 = 2$$

$$2^2 = 4$$

$$2^4 = 16$$

$$2^8 = 256$$

$$2^{16} = 256^2 = 460 \pmod{561}$$

$$2^{32} = 460^2 = 103 \pmod{561}$$

$$2^{64} = 103 \pmod{561}$$

$$2^{128} = 103^2 = 361 \pmod{561}$$

$$2^{256} = 361^2 = 466 \pmod{561}$$

$$2^{512} = 466^2 = 103 \pmod{561}$$

since, $560 = 512 + 32 + 16$

$$2^{560} = 2^{512} \cdot 2^{32} \cdot 2^{16}$$

$$= 103 \times 103 \times 466$$

$$= 1 \pmod{561}$$

Interpretation

Although $561 = 3 \times 11 \times 17$ is composite

$$2^{560} = 1 \pmod{561}$$

Thus, the simple Fermat test with base 2 incorrectly suggests that 561 may be prime. This illustrates why Carmichael numbers can fool the Fermat tests and why stronger test such as Miller-Rabin are needed.

Question-5: Compare deterministic (AKS) and probabilistic (Miller-Rabin) primality testing in terms of asymptotic complexity, error probability, and practical relevance to RSA key generation at 2048-bit and 4096-bit key sizes.

⇒ AKS vs Miller-Rabin.

Feature	Miller-Rabin	AKS
Type	Probabilistic	Deterministic
Complexity	About $O(k^3)$ for a k -bit number per base	polynomial, roughly $O(k^6)$ in the original form.
Error	Small probability of errors with t independent bases $< 4^{-t}$ for an odd composite.	Zero error
Practical use	Widely used for RSA prime generation.	Mainly theoretical; rarely used in practical RSA generation.

For 2048 & 4096-bit RSA, Miller-Rabin is preferred because it is extremely fast and its error probability can be made negligible by using multiple bases.

$$2^{64} = 103^4 \pmod{561}$$

$$2^{128} = 103^8 = 361 \pmod{561}$$

$$2^{256} = 361^2 = 466 \pmod{561}$$

$$2^{512} = 466^2 = 103 \pmod{561}$$

since, $560 = 512 + 32 + 16$

$$2^{560} = 2^{512} \cdot 2^{32} \cdot 2^{16}$$

$$= 103 \times 103 \times 466$$

$$= 1 \pmod{561}$$

Interpretation

Although $561 = 3 \times 11 \times 17$ is composite

$$2^{560} = 1 \pmod{561}$$

Thus, the simple Fermat test with base 2 incorrectly suggests that 561 may be prime. This illustrates why Carmichael numbers can fool the Fermat tests and why stronger test such as Miller-Rabin are needed.

Question no: 06 - Derive the Extended Euclidean algorithm formally (as square of linear combinations), then use it to compute $\gcd(240, 46)$ together with the Bezout coefficients x, y such that $240x + 46y = \gcd(240, 46)$.



The Euclidean Algorithm gives,

$$r_{i-2} = q_i r_{i-1} + r_i$$

The Extended Euclidean Algorithm (EEA) expresses every remainder as a linear combination of the original numbers:

$$r_i = s_i(240) + t_i(46)$$

For 240 and 46

$$240 = 5(46) + 10$$

$$46 = 4(10) + 6$$

$$10 = 1(6) + 4$$

$$6 = 1(4) + 2$$

$$4 = 2(2) + 0$$

Therefore

$$\gcd(240, 46) = 2$$

Back substitution,

$$2 = 6 - 4$$

$$= 6 - (10 - 6)$$

$$= 2(6) - 10$$

$$= 2(46 - 4(10)) - 10$$

$$= 2(46) - 9(10)$$

$$= 2(46) - 9(240 - 5(46))$$

$$= 47(46) - 9(240)$$

$$\text{Thus, } 240(-9) + 46(47) = 2$$

Hence the Bezout coefficients are

$$x = -9, \quad y = 47$$

$$\text{and } \boxed{\gcd(240, 46) = 2}$$

Question - 07: state and prove that Chinese Remainder Theorem for pairwise coprime moduli. Apply it to solve: $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$, $x \equiv 4 \pmod{7}$, $x \equiv 5 \pmod{11}$

⇒

Theorem: If $m_1, m_2, \dots, m_k$ are pairwise coprime then the system

$$x \equiv a_i \pmod{m_i}$$

has a unique solution modulo

$$M = \prod_{i=1}^k m_i, \quad M_i = \frac{M}{m_i}$$

Since $\gcd(m_i, m_j) = 1$, M_i has an inverse y_i modulo m_i :

$$M_i y_i \equiv 1 \pmod{m_i}$$

Then

$$x = \sum_{i=1}^k a_i M_i y_i$$

satisfies every congruence, proving existence.

If x and x' are two solutions, then

$$m_i \mid (x - x')$$

for every i since the moduli are pairwise coprime.

$$M \mid (x - x')$$

Therefore $x \equiv x' \pmod{M}$.

proving uniqueness.

Apply CRT:

Given, $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$, $x \equiv 4 \pmod{7}$.

$$x \equiv 5 \pmod{11}$$

$$M = 3 \times 5 \times 7 \times 11 = 1155.$$

$$M_1 = 385, M_2 = 231, M_3 = 165, M_4 = 105.$$

Their inverse are:

$$385^{-1} \equiv 1 \pmod{3}$$

$$231^{-1} \equiv 1 \pmod{5}$$

$$165^{-1} \equiv 4 \pmod{7}$$

$$105^{-1} \equiv 2 \pmod{11}$$

$$\begin{aligned} \text{Hence, } x &\equiv 2(385)^{(1)} + 3(231)(1) + 4(165)(4) + 5(105)(2) \\ &= 770 + 693 + 2640 + 1050 = 5153. \end{aligned}$$

Reducing modulo 1155,

$$5153 - 4(1155) = 533.$$

$$x \equiv 533 \pmod{1155}$$

$$\therefore x = 533 + 1155K, \quad K \in \mathbb{Z}$$

Question-8: Derive the CRT-based RSA decryption/signing optimization from first principles (computing d_p , d_q and q_{inv}) and demonstrate the $\sim 4x$ speed up with a fully worked numerical example using small primes p, q .

$\Rightarrow$ Let,

$$\boxed{n = pq}, \quad \boxed{\phi(n) = (p-1)(q-1)}$$

and RSA private exponent be d .

Instead of computing $m = c^d \bmod n$.

we compute separately modulo p and q .

1. CRT parameters,

$$d_p = d \bmod (p-1)$$

$$d_q = d \bmod (q-1)$$

$$q_{inv} = q^{-1} \bmod p$$

Then, $m_p = c^{d_p} \bmod p$, $m_q = c^{d_q} \bmod q$.

Recombine using CRT,

$$h = q_{inv} (m_p - m_q) \bmod p$$

$$m = m_q + qh$$

Numerical example:-

$$p = 11, q = 17.$$

$$n = 187.$$

$$\text{Let, } d = 23, \quad c = 100.$$

$$\text{calculate, } dp = 23 \bmod 10 = 3.$$

$$dq = 23 \bmod 16 = 7.$$

$$\text{Find } q^{-1} \bmod 11 = 2.$$

$$\text{because } 17(2) = 34 \equiv 1 \bmod (11)$$

$$\text{now, } mp = 100^3 \bmod 11$$

$$\text{since, } 100 \equiv 1 \bmod (11)$$

$$mp = 1.$$

$$\text{since } 100 \equiv 15 \bmod (17)$$

$$15^7 \equiv 8 \bmod (17).$$

$$mp = 1, \quad mq = 8.$$

$$h = 2(1 - 8) \bmod 11, \quad h = -14 \bmod 11 = 1.$$

$$m = 8 + 17(8) = \boxed{144}$$

Direct RSA computation gives,

$$100^{23} \bmod 187 = \boxed{144}$$

So both methods agree.

Question-09: Explain the Bellcore fault-injection attack on RSA-CRT signatures, how a single faulty modular computation during signing leaks a factor of n , and what countermeasures prevent it.

⇒ In RSA-CRT signing

$$s_p = m^d \pmod{p}, \quad s_q = m^d \pmod{q}$$

and the two results are combined using CRT to produce signature s .

Bellcore attack: If an attacker induces a fault in one computation, say s_q , the device produces a faulty signature s'' .

$$s''^e = m \pmod{n}$$

$$\gcd(s''^e - m, n)$$

reveals one of the secret factors p or q .
Once one factor is known,

$$q = \frac{n}{p}$$

and the attacker can reconstruct the RSA private key.

Countermeasures:-

1. Verify signature before releasing it:

$$s^e \bmod n = m.$$

If verification fails, discard the signature.

2. Use fault detection/error checks during CRT computation.

3. Use blinding/randomization to make fault attack harder.

Question - 10:- Compute a modular inverse using both the Extended Euclidean Algorithm and Fermat's Little Theorem base exponentiation and discuss the trade offs between the two approaches for large moduli,

⇒ suppose we need to find the modular inverse of

$$a = 3 \pmod{11}$$

$$\text{find } 3^{-1} \pmod{11}.$$

1. EEA:

$$\text{we use } 11 = 3(3) + 2$$

$$3 = 2(1) + 1.$$

Back substitution,

$$1 = 3 - 2$$

$$\Rightarrow 1 = 3 - (11 - 3(3))$$

$$\Rightarrow 1 = 4(3) - 11$$

$$\Rightarrow 4(3) = 1 \pmod{11}.$$

$$3^{-1} \equiv 4 \pmod{11}$$

2. FLT:

For a prime modulus p , $a^{p-1} \equiv 1 \pmod{p}$

$$a^{-1} = a^{p-2} \pmod{p}$$

For, $a = 3$, $p = 11$.

$$3^{-1} = 3^9 \pmod{11}$$

using modular exponentiation: $3^4 = 9$

$$\Rightarrow 3^4 = 9^4 = 81 = 4 \pmod{11}$$

$$\Rightarrow 3^8 = 4^4 = 16 = 5 \pmod{11}$$

$$\Rightarrow 3^9 = 5 \times 3 = 15 = 4 \pmod{11}$$

Thus, $\boxed{3^{-1} = 4 \pmod{11}}$

Trade-offs for Large Moduli:-

Method

Advantage

Limitation

• EEA

works for any a ,
 m when $\gcd(a, m) = 1$;
usually efficient.

Requires Euclidean
divisions.

• FLT expo-
nentiation.

Simple & convenient
when modulus is
prime, uses modular
exponentiation.

Requires prim
modulus and
exponentiati
up to $p-2$

Question-11: Design and fully execute a playfair cipher encryption and decryption of a phrase of your choosing. Present the complete 5×5 key matrix as a table, and show every digraph substitution step.

⇒ Construct the 5x5 key matrix = [2x2] and 1
keyword: MONARCHY

Remove repeated letter and fill the remaining alphabet combining I/J.

unique keyword letters: MONARCHY

M	O	N	A	R
C	H	Y	B	D
E	F	G	I	K
L	P	Q	S	T
U	V	W	X	Z

plaintext:

Pharmacist
meet me At the Library tonight.

split into digraphs:

meet me at the library tonight

Encryption rules:

- same row $\rightarrow$ replace each letter by the letter immediately to its right.
- same column $\rightarrow$ replace below it.
- Rectangle $\rightarrow$ replace by the letter in the same row but the other's letter's column.

me	et	me	at	th	el	ib	ra	ry	to	ni	gh	tx
cl	kl	cl	rs	pd	lu	si	mr	nd	pr	ag	fy	sz

Decryption rule:

- same row $\rightarrow$ move one position left.
- same column $\rightarrow$ " " " " up.
- Rectangle $\rightarrow$ use the same rectangle rule.

CL	KL	CL	RS	PD	LU	SI	MR	ND	PR	AG	FY	SZ
ME	ET	ME	AT	TH	EL	IB	RA	RY	TO	NI	GH	TX.

question - 12:- critically analyze the structural weaknesses of the playfair cipher against digraph frequency analysis, and explain why it remains vulnerable to known plaintext attacks despite being far stronger than monoalphabetic substitution.

⇒ structural weaknesses of playfair cipher:-

The playfair cipher is stronger than a monoalphabetic substitution because it encrypts pairs of letters (digraphs) rather than individual letters. However, it is still vulnerable to statistical and known plaintext attacks.

- Digraph frequency analysis:- common pairs like TH, HE, IN can produce noticeable ciphertext pattern.

- Repeated digraphs:- The same plaintext pair always produces the same ciphertext pairs.

- Known plaintext attack:- several known pairs give constraints on the key square, allowing an attacker to gradually reconstruct it.

Question - 13: Derive the trace two full rounds of a simplified Feistel-structure cipher (DES-like), showing $L_0, R_0 \rightarrow L_1, R_1 \rightarrow L_2, R_2$ symbolically, then with the numeric values $L_0 = 0xAAAAAAAA, R_0 = 0xFOFOFOFO, K_1 = 0x0F0F0F0F, K_2 = 0x33333333$. Using XOR as the round function.

⇒ For a Feistel cipher, the round equations are:

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

Here the simplified round function is,

$$F(R, K) = R \oplus K$$

1. Symbolic Trace

Round 1: $L_1 = R_0, R_1 = L_0 \oplus (R_0 \oplus K_1)$

Round 2: $L_2 = R_1, R_2 = L_1 \oplus (R_1 \oplus K_2)$

Thus, $L_0, R_0 \rightarrow L_1, R_1 \rightarrow L_2, R_2$

2. Numerical calculation:-

Given,

$$L_0 = AAAAAAAAAA$$

$$R_0 = FOFOFOFO$$

$$K_1 = 0F0F0F0F$$

$$K_2 = 33333333$$

Round-1:

$$F(R_0, k_1) = F0F0F0F0 \oplus 0F0F0F0F$$

$$= FFFFFFFF$$

Therefore,

$$L_1 = R_0 = F0F0F0F0$$

$$R_1 = L_0 \oplus F(R_0, k_1)$$

$$= AAAAAAAAAA \oplus FFFFFFFF$$

$$= 55555555$$

So, $\boxed{L_1, R_1 = F0F0F0F0, 55555555}$

Round-2:

$$F(R_1, k_2) = 55555555 \oplus 33333333$$

$$= 66666666$$

Therefore,

$$L_2 = R_1 = 55555555$$

$$R_2 = L_1 \oplus F(R_1, k_2)$$

$$= F0F0F0F0 \oplus 66666666$$

$$= 96969696$$

$$\therefore L_2 = 55555555, R_2 = 96969696$$

Question - 14: Formally define the avalanche effect in block ciphers. Using a toy 8 bit substitution permutation network of your own construction, demonstrate how flipping a single input bit changes at least half of the bits, and explain why this property is essential for cryptographic security.

⇒ The avalanche effect means that a small change in the plaintext or key, such as flipping one bit, should cause a large and unpredictable change in the ciphertext. Ideally, about 50% of the output bits should change.

Toy 8 bit SPN example:-

Suppose our 8 bit SPN produces,

$$P = 10110010$$

and after substitution and permutation,

$$C = 01101101$$

Now flip only the first input bit.

$$P' = 00110010$$

After passing through the same SPN,

$$C' = 11010010$$

Compare,

$$C = 01101101$$

$$C' = 11010010$$

changed bits:- 1, 2, 4, 5, 7, 8.

so 6 out of 8 bits changed,

$$\frac{6}{8} \times 100 = 75\%$$

Thus flipping one input bit changed 75% of the output bits, which demonstrate a strong avalanche effect.

Importance:-

- Hides the relationship between plaintext and ciphertext.
- prevents attackers from predicting how an input change affects the ciphertext.
- makes statistical and differential attacks more difficult.

One bit input change $\rightarrow$ many output bit changes.

Question-15:- Evaluate the brute-force resistance of an 8-character alphanumeric password (62-character alphabet) versus a 6 digit PIN, under a modern GPU cracking rate of 10^9 guesses/second. Compute worst-case and expected-case cracking times for both, and discuss the implications for password policy design.

⇒ Given cracking rate:

$$10^9 \text{ guesses/second}$$

1. 8 password-character alphanumeric.

$$N = 62^8 = 218,340,105,584,896.$$

Worst-case:

$$T = \frac{62^8}{10^9} = 218,340.15 \text{ s.}$$
$$\approx 60.65 \text{ hours} \approx 2.53 \text{ days.}$$

$$\text{expected case: } \frac{2.53}{2} \approx 1.26 \text{ days.}$$

2. 6 digit pin, $N = 10^6 = 1,000,000.$

$$\text{Worst case: } T = \frac{10^6}{10^9} = 0.001 \text{ s} = 1 \text{ ms.}$$

$$\text{expected case: } 0.5 \text{ ms.}$$

<u>Credential</u>	<u>total possibilities</u>	<u>Worst case</u>	<u>expected case</u>
8 char alpha numeric	62^8	2.53 days	1.26 days
6 digit PIN	10^6	4ms	0.5ms

Implication:

A 6 digit PIN has a very small brute-force search space compared with an 8 characters alphaneumeric password. Longer password provides substantially greater brute force resistance.

Question-10:

⇒ 1. ECB:

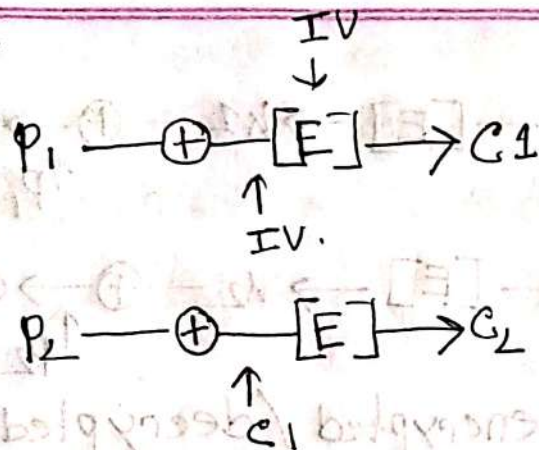
$$P_1 - [E] \rightarrow C_1$$

$$P_2 - [E] \rightarrow C_2$$

$$P_3 - [E] \rightarrow C_3$$

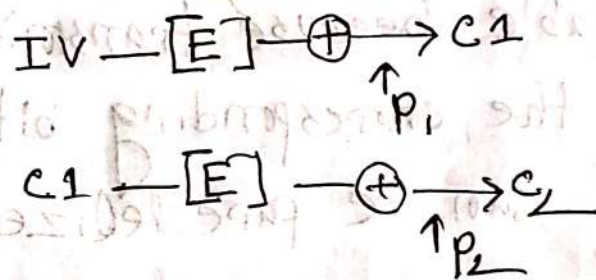
A bit error affects only its corresponding block. However, ECB reveals identical plaintext patterns.

2. CBC:-



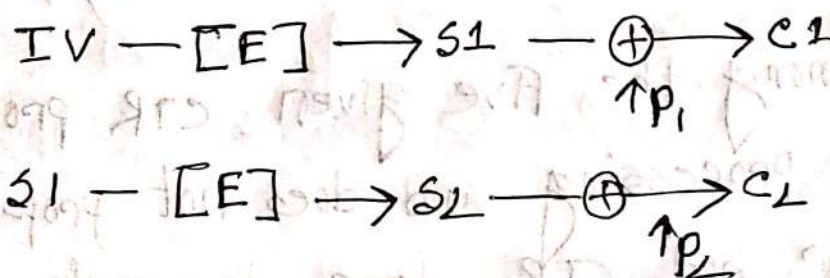
A ciphertext bit error affects the current plaintext block and the corresponding bit of the next block.

3. CFB:-



A bit error propagates to the current block and may affect the next block.

4. OFB:-



A transmission bit error affects only the corresponding bit, it does not propagate.

5. CTR:

$$\text{nonce}+1 \xrightarrow{[E]} k_1 \xrightarrow{\oplus \uparrow p_1} c_1$$

$$\text{nonce}+2 \xrightarrow{[E]} k_2 \xrightarrow{\oplus \uparrow p_2} c_2$$

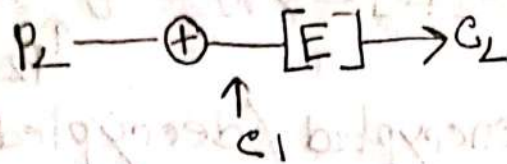
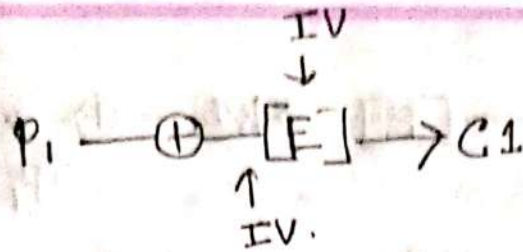
Blocks can be encrypted / decrypted independently and in parallel. A bit error affects only the corresponding bit.

① Live video over a lossy network $\rightarrow$ CTR
CTR is suitable because transmission errors affect only the corresponding bit and encryption / decryption can be parallelized.

② Static disk image $\rightarrow$ XTS-AES would normally be preferred in real disk encryption.

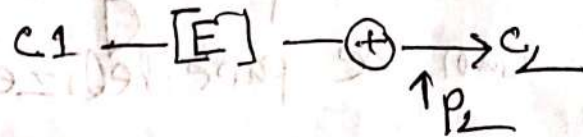
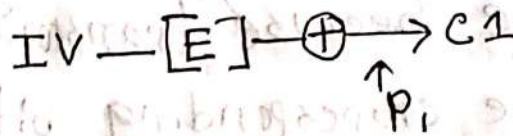
$\Rightarrow$ Among the five given, CTR provides parallel processing and does not propagate bit errors, but CTR alone does not provide authentication and has important nonce management requirements.

2. CBC:-



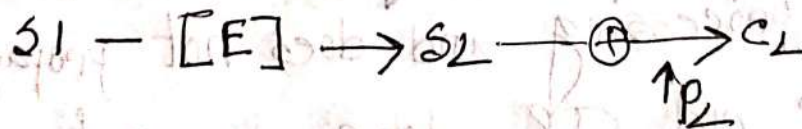
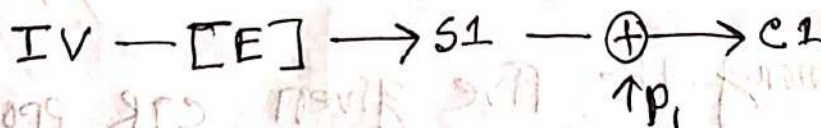
A ciphertext bit error affects the current plaintext block and the corresponding bit of the next block.

3. CFB:-



A bit error propagates to the current block and may affect the next block.

4. OFB:-



A transmission bit error affects only the corresponding bit, it does not propagate.

5. CTR:

$$\text{nonce}+1 \xrightarrow{[E]} k_1 \xrightarrow{\oplus \uparrow p_1} c_1$$

$$\text{nonce}+2 \xrightarrow{[E]} k_2 \xrightarrow{\oplus \uparrow p_2} c_2$$

Blocks can be encrypted / decrypted independently and in parallel. A bit error affects only the corresponding bit.

① Live video over a lossy network $\rightarrow$ CTR
CTR is suitable because transmission error affect only the corresponding bit and encryption / decryption can be parallelized.

② static disk image $\rightarrow$ XTS-AES would normally be preferred in real disk encryption.

$\Rightarrow$ Among the five given, CTR provides parallel processing and does not propagate bit error, but CTR alone does not provide authentication and has important nonce-management requirements.

Question-18: Derive the AES-128 Rijndael key schedule and compute the first two round key (K_1, K_2) from given 128 bit cipher key of your choosing, showing the Rotword, subword and Rcon steps explicitly.

⇒ chosen the 128 bit cipher key

2B7E151628AED2A6ABF7158809CF4F3C

Divide it into four 32 bit words:

$W_0 = 2B7E1516$, $W_1 = 28AED2A6$

$W_2 = ABF71588$, $W_3 = 09CF4F3C$

For AES-128:

$$W_i = W_{i-4} \oplus T(W_{i-1})$$

where T consists of Rotword $\rightarrow$ subword $\rightarrow$ Rcon

Round key K_1 ,

$$\text{Rotword}(W_3) = CF4F3C09$$

Apply AES S-box,

$$\text{subword}(CF4F3C09) = 8A84EB0.$$

For round 1, $Rcon_1 = 01000000$.

$$\begin{aligned} \text{Therefore, } T(W_3) &= 8A84EB01 \oplus 01000000 \\ &= 8B84EB01. \end{aligned}$$

Now,

$$W_4 = W_0 \oplus 8B84EB01 = 2B7E1516 \oplus 8B84EB01 \\ = A0FAFE17.$$

$$W_5 = W_1 \oplus W_4 = 28AED2A6 \oplus A0FAFE17 = 88542CB1$$

$$W_6 = W_2 \oplus W_5 = 23A33939$$

$$W_7 = W_3 \oplus W_6 = 2A6C7605.$$

$$K_1 = A0FAFE1788542CB123A339392A6C7605$$

$$K_2 = F2C3F297AA81398590B2AA17365C415$$

Question-19: Explain the mixcolumns transformation as polynomial multiplication over $GF(2^8)$ modulo $x^8 + x^4 + x^3 + x + 1$. Work through the full multiplication of one input column by the fixed mixcolumns matrix, showing the $GF(2^8)$ arithmetic for each byte.

⇒ In AES, each column is treated as a polynomial over $GF(2^8)$ with irreducible polynomial.

$$m(x) = x^8 + x^4 + x^3 + x + 1$$

The fixed mixcolumns matrix is,

$$M(x) = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

Take one input column.

$$\begin{bmatrix} db \\ 13 \\ 53 \\ 45 \end{bmatrix}$$

Then,

$$b_0 = (02 \cdot db) \oplus (03 \cdot 13) \oplus 53 \oplus 45.$$

$$b_1 = db \oplus (02 \cdot 13) \oplus (03 \cdot 53) \oplus 45$$

$$b_2 = db \oplus 13 \oplus (02 \cdot 53) \oplus (03 \cdot 45)$$

$$b_3 = 76 \oplus 13 \oplus 53 \oplus 8a \oplus (02 \cdot 45)$$

GF(2⁸) multiplications,

$$b_0 = ad \oplus 35 \oplus 53 \oplus 45 = 8e$$

$$b_1 = ab \oplus 26 \oplus f5 \oplus 45 = 4d$$

$$b_2 = db \oplus 13 \oplus 2b \oplus cf = a1$$

$$b_3 = 76 \oplus 13 \oplus 53 \oplus 8a = bc$$

output is,

$$\begin{bmatrix} 8e \\ 4d \\ a1 \\ bc \end{bmatrix}$$

Question - 20:-

AES itself is mathematically secure, but implementation can leak information through side channels.

1. Timing / Cache Attack:-

Table based AES implementation use s-box look-up tables. Different secret-dependent memory accesses can cause different cache hit/miss times. An attacker can collect many timing and infer the secret key.

Countermeasure:- use a constant-time s-box, such as bitsliced implementation, where execution time and memory access do not depend on secret data.

Trade off:- usually requires more CPU operations and may increase code complexity.

2. Differential Power Analysis (DPA):-

The device's power consumption varies with intermediate AES values. By collecting many power traces and statistically correlating

them with key guesses, an attacker may recover the key.

Countermeasure: masking $\rightarrow$ randomize intermediate values, $\boxed{r \rightarrow r \oplus r}$.

Trade off: masking requires extra operation, randomness and sometimes additional memory, reducing performance.

Question-21: Derive the RSA key-generation, encryption and decryption procedures from Euler's Theorem, and prove correctness of decryption for the general case, including the case $\gcd(m, n) \neq 1$.

$\Rightarrow$ 1. key generation:

choose two large primes p, q .

$$n = pq, \quad \phi(n) = (p-1)(q-1).$$

choose e such that,

$$\gcd(e, \phi(n)) = 1.$$

Find d satisfying $ed = 1 \pmod{\phi(n)}$.

public key: (e, n)

private key (d, n)

2. Encryption & Decryption:-

For message m :

$$C = M^e \bmod n$$

and $M' = C^d \bmod n = M^{ed} \bmod n$

since, $\boxed{ed = 1 + k\phi(n)}$

euler's theorem gives, $M^{\phi(n)} = 1 \pmod{n}$,

when $\gcd(M, n) = 1$. Therefore,

$$M^{ed} = M^{1+k\phi(n)} = M (M^{\phi(n)})^k \equiv M \pmod{n}$$

Hence, $M' = M$.

case: $\gcd(M, n) \neq 1$.

Let $n = pq$, where p, q are distinct prime

If $p \mid M$, then, $M \equiv 0 \pmod{p}$

$$M^{ed} \equiv 0 \equiv M \pmod{p}$$

Since $\gcd(M, q) = 1$,

$$M^{q-1} = 1 \pmod{q}$$

$$ed = 1 \pmod{q-1}$$

For all M , including $\gcd(M, n) \neq 1$.

$$M^{ed} \equiv M \pmod{p}$$

Question-022: Given primes $p=11$, $q=13$, generate a full RSA keypair (choosing a valid public exponent, e) then sign a message hash $H(M)=23$ and verify the signature, showing all intermediate modular exponentiation steps.

⇒

Given, $p=11$, $q=13$, $H(M)=23$.

1. key generation,

$$n = pq = 11 \times 13 = 143.$$

$$\phi(n) = 10 \times 12 = 120.$$

choose, $e=7$.

since, $\boxed{\gcd(7, 120) = 1}$

Find d ,

$$ed \equiv 1 \pmod{120}$$

$$\Rightarrow 7 \times d \equiv 1 \pmod{120}$$

$$\Rightarrow 7 \times 103 \equiv 1 \pmod{120}.$$

$$\therefore d = 103.$$

public key = $(7, 143)$

private key = $(103, 143)$.

2. signing:

The signature is,

$$s = H(m)^d \pmod{n}$$

$$s = 23^{103} \pmod{143}.$$

Using square and multiply

$$23^1 = 23, \quad 23^2 = 529 \equiv 100.$$

$$23^4 \equiv 100^2 = 10000 \equiv 12.$$

$$23^8 = 12^2 = 144 \equiv 1.$$

$$23^{16} = 1^2 = 1 \equiv 1.$$

$$23^{32} = 1, \quad 23^{64} = 1.$$

$$103 = 64 + 32 + 4 + 2 + 1$$

$$\therefore 23^{103} = 1 \times 1 \times 12 \times 100 \times 23 \pmod{143}$$
$$= 78$$

3. signature verification:

$$s^e \pmod{n} = 78^7 \pmod{143}$$

$$78^2 = 6084 \equiv 41$$

$$78^4 = 41^2 = 1681 \equiv 107.$$

$$78^7 = 78^4 \times 78^2 \times 78 = 23.$$

since, $s^e \pmod{n} = 23 = H(m)$

the signature is valid.

Question - 23:-

In Esp tunnel mode, the overhead consists of the Esp header, authentication data, padding, and the new outer Ip header.

Let,

- p = payload size
- H = Esp. header size
- A = Authentication-data size
- pad = padding
- I = outer Ip header size

Then, $O = H + A + pad + I$

and the overhead percentage is,

$$\text{overhead \%} = \frac{O}{p+O} \times 100$$

For given values,

$$p = 1200, H = 8, A = 16, pad = 2$$

Assuming a 20 bit IPv4 outer header,

$$O = 8 + 16 + 2 + 28 = 46 \text{ bytes.}$$

Total transmitted size, $1200 + 46 = 1246$ bytes.

Therefore, $\text{overhead \%} = \frac{46}{1246} \times 100 = 3.69 \%$

scaling: For small IoT packet, fixed ESP/IP headers form a large percentage of the packet, so overhead is relatively high. For large payloads, the fixed overhead is spread over more data, so the percentage becomes much smaller.

small packet $\rightarrow$ high overhead %

large " $\rightarrow$ low " %

question-24:

Execute a full Diffie-Hellman key exchange with $p=23$, $g=5$, private keys $a=6$ (Aleya), $b=15$ (Badol), confirming both parties derive the same shared secret. Then explain man-in-the-middle vulnerability of unauthenticated DH and describe how an authenticated variant mitigates it.

$\Rightarrow$ Given,

$p=23$, $g=5$

Aleya's private key, $a=6$

Badol's " " , $b=15$

1. public keys:

Aleya computes:

$$A = g^a \bmod p = 5^6 \bmod 23 = 8.$$

Badol computes:

$$B = g^b \bmod p = 5^{15} \bmod 23 =$$

$$\dots 5^4 = 25 \bmod 23 = 2$$

$$5^8 = 2^4 \bmod 23 = 4$$

$$5^8 = 4^2 \bmod 23 = 16.$$

$$5^{15} = 5^8 \cdot 5^4 \cdot 5^2 \cdot 5$$

$$= 16 \times 4 \times 2 \times 5 \bmod p$$

$$= 19.$$

$$\therefore B = 19.$$

2. shared secret:

Aleya:

$$K = B^a \bmod p = 19^6 \bmod 23 = 2$$

Badol:

$$K = A^b \bmod p = 8^{15} \bmod 23 = 2.$$

$$8^4 = 16, 8^4 = 16^2 = 3, 16^8 = 3^2 = 9$$

$$\therefore 8^{15} = 8^8 \cdot 8^4 \cdot 8^2 \cdot 8$$

$$= 9 \times 3 \times 16 \times 8 = 2$$

$$\boxed{K_{Aleya} = K_{Badol} = 2}.$$

MITM vulnerability:- Unauthenticated DH does not authenticate the communicating parties. An attacker can intercept A and B, replace them with their own public values, and establish separate keys with Aleya and Badol.

Aleya $\longleftrightarrow$ Attacker $\longleftrightarrow$ Badol.

The attackers can decrypt, modify and re-encrypt message.

DH provides secrecy,
authenticated DH provides secrecy + authentication

Question - 25:- Compare the computational cost of establishing a shared key via RSA key transport versus Diffie-Hellman key agreement at an equivalent security level. Discuss why forward secrecy favors (EC) DHE over RSA key transport in modern TLS.

⇒ RSA key transport vs Diffie Hellman.

<u>Features</u>	<u>RSA key Transport</u>	<u>Diffie Hellman</u>
• 3072-bit RSA/DH.	• Large modular exponentiation RSA encryption is relatively fast, but private key operation is costly.	• Similar large computationally heavier than ECDH.
• 256 bit ECDH		• much smaller keys and generally much faster, especially for key agreement.
• Forward Secrecy	NO	YES.
• Modern TLS	Legacy RSA key transport is not used in current TLS 1.3.	ECDH is the standard approach.

Why forward secrecy favors (EC)DHE? In RSA key transport, the client encrypts the session secret using the server's long term RSA public key. If the server's private RSA key is later compromised, previously recorded sessions can potentially be decrypted.

Question 26: In an ElGamal schema with $p = 467$, $g = 2$, private key $k = 127$, message hash $H(m) = 127$ and nonce $r = 213$ (with $\gcd(k, p-1) = 1$) compute public key, generate the signature (r, s) and verify it.

⇒ Given,

$$p = 467, g = 2, x = 127, H(m) = 100, k = 213$$

Since $\gcd(213, 466) = 1$, the nonce is valid.

1. public key:

$$Y = g^k \bmod p$$

$$Y = 2^{127} \bmod 467$$

power calculation,

$$2^1 = 2, 2^2 = 4, 2^4 = 16, 2^8 = 256$$

$$2^{16} = 150, 2^{32} = 52$$

$$2^{64} = 369$$

$$\therefore 2^{127} = (369 \times 52 \times 150 \times 256 \times 16 \times 4 \times 2) \bmod 467 = 132$$

so the public key $(p, g, Y) = (467, 2, 132)$.

2. Generate signature (r, s)

First, $r = g^k \pmod{p}$.

$$r = 2^{213} \pmod{467} \\ = 29.$$

Find $k^{-1} \pmod{466}$.

$$213^{-1} = 431 \pmod{466}$$

ElGamal signature formula,

$$s = k^{-1} (H(m) - kr) \pmod{p-1}$$

$$s = 431 (100 - 127 \times 29) \pmod{466} \\ = 51.$$

$$\boxed{(r, s) = (29, 51)}$$

3. Verification:

$$g^{H(m)} = y^r r^s \pmod{p}$$

$$L.H.S = g^{200} \pmod{467} = 189$$

$$R.H.S = 132^{29} \times 29^{51} \pmod{467} = 189,$$

$$g^{H(m)} = y^r r^s \pmod{p}$$

and the signature is valid.

Question - 27 :- Prove algebraically that the ElGamal signature verification equation $g^{H(M)} = y^r \cdot r^s \pmod{p}$ holds for a correctly generated signature, starting from the definition $s \equiv k^{-1} (H(M) - kr) \pmod{p-1}$

⇒ Given,

$$y = g^k \pmod{p}$$

$$s = k^{-1} (H(M) - kr) \pmod{p-1}$$

Multiplying both sides by k ,

$$ks = H(M) - kr \pmod{p-1}$$

$$\Rightarrow H(M) = kr + ks \pmod{p-1}$$

Now raise g to both sides

$$g^{H(M)} = g^{kr + ks} \pmod{p}$$

$$\Rightarrow g^{H(M)} = (g^k)^r \cdot (g^k)^s \pmod{p}$$

Since, $y = g^k \pmod{p}$ and $r = g^k \pmod{p}$

$$\therefore \boxed{g^{H(M)} = y^r \cdot r^s \pmod{p}}$$

Hence, the ElGamal signature verification

equation is proved $g^{H(M)} = y^r \cdot r^s \pmod{p}$

question-28: on the elliptic curve $y^2 = x^3 + x + 1 \pmod{23}$ verify that $P = (3, 10)$ lies on the curve, compute $2P$ via point doubling and compute $P+Q$ for $Q = (9, 7)$ showing the full modular arithmetic derivation of the slope in each case.

⇒

Given the elliptic curve $y^2 = x^3 + x + 1 \pmod{23}$.

1. verification that $P = (3, 10)$ lies on the curve.

$$\text{L.H.S} = y^2 = 10^2 = 100 \equiv 8.$$

$$\text{R.H.S} = x^3 + x + 1 \equiv 8.$$

since $\boxed{\text{L.H.S} = \text{R.H.S} \pmod{23}}$ $P(3, 10)$ lies on the curve

2. compute $2P$ (point doubling).

for $P(x_1, y_1) = (3, 10)$

$$\text{slope } m :- m = \frac{3x_1^2 + a}{2y_1} \pmod{23} = \frac{27+1}{20} = \frac{1}{4} \pmod{23}$$

$$\text{To find } 4^{-1} \pmod{23} :- 4 \times 6 = 24 \equiv 1 \pmod{23}.$$

$$\text{So, } 4^{-1} = 6.$$

$$m = 1 \times 6 = 6 \pmod{23}.$$

coordinates $(x_3, y_3) = 2P$.

$$x_3 = m^2 - 2x_1 \pmod{23} = 36 - 6 = 30 = 7.$$

$$y_3 = m(x_1 - x_3) - y_1 \pmod{23} = 6(3 - 7) - 10 = -34 \pmod{23} = 12$$

Result $2p = (7, 12)$.

3. compute $p+q$ for $q = (9, 7)$ for, $p(3, 10)$ $q(9, 7)$

slope m:

$$m = \frac{y_2 - y_1}{x_2 - x_1} \pmod{23} = \frac{-3}{6} = -\frac{1}{2} = \frac{22}{2} = 11.$$

coordinates $(x_3, y_3) = p+q$

$$x_3 = m^2 - x_1 - x_2 \pmod{23}$$

$$= 11^2 - 3 - 9 = 121 - 12 = 109 = 17.$$

$$y_3 = m(x_1 - x_3) - y_1 \pmod{23}$$

$$= 20.$$

$$\therefore (p+q) = (17, 20).$$

Question - 29:- On the curve $y^2 = x^3 + 7x + 10 \pmod{31}$ with base point $G = (2, 5)$ of order $n = 19$ and private key $d = 9$. compute the public key $Q = dG$ generate an ECDSA signature (r, s) for $H(m) = 8$ using nonce $k = 3$ and verify it against.

$\Rightarrow$ There is a problem with the given question the stated base point $G = (2, 5)$ is not on the given curve.

check, $5^4 = 625 = 25$.

but, $x^3 + 7x + 10 = 2^3 + 7(2) + 10 = 32$

modulo 37, $25 \neq 32 \pmod{37}$

Therefore, $G = (2, 5) \notin E$

Given parameters are mathematically inconsistent.

Question-30:

RSA, ElGamal and ECC security comparison -

<u>Algorithm</u>	<u>Approx. key size</u>	<u>security level</u>
RSA	3072 bits	≈ 128 bit.
ElGamal	3072 bits	≈ 128 bit
ECC	256 bits	≈ 128 bit

Thus, 256-bit ECC $\approx$ 3072-bit RSA/ElGamal ≈ 128 bit security

Why ECC is preferred for IoT and smart cards.

$\Rightarrow$ ECC provide equivalent security with much smaller key.

- Less memory and storage.
- Less computation and power consumption.
- smaller communication bandwidth.
- Faster key generation and signatures.
- Lower resources requirements.

Question - 31:-

Let, $H: \{0,1\}^* \rightarrow \{0,1\}^n$ be an n bit cryptographic hash function.

1. Preimage Resistance :- Given a hash value h , it should be computationally infeasible to find any message m such that,

$$H(m) = h.$$

expected work : 2^n

2. Second - preimage Resistance :- Given a message m_1 , $m_2 \neq m_1$, $H(m_2) = H(m_1)$

3. Collision Resistance :- It should be infeasible to find any two distinct message m_1, m_2 such that

$$m_1 \neq m_2, H(m_1) = H(m_2)$$

Birthday Attack:- For an n -bit hash, there are $N = 2^n$.

$$k = O(2^{n/2})$$

Thus the collision attack requires $2^{n/2}$.

Why 256 bit Hashes?

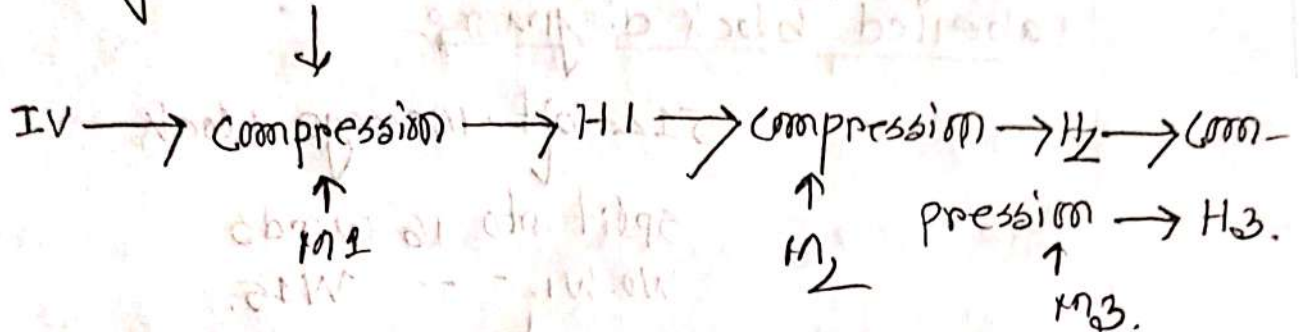
theoretically find a collision $\frac{2^{256}}{2} = 2^{128}$ work can

$2^{256/2} = 2^{128}$
256-bit output $\rightarrow$ much stronger for security

Question-32:-

Merkle-Damgard Construction & Length-Extension attack

⇒ message : m_1 m_2 m_3 .



Final Hash = H_3 .

- Merkle-Damgard:- The message is divided into fixed-size blocks. Each block is processed with a compression function

$$H_i = CF(H_{i-1}, m_i)$$

- Long-extension attack:- The final hash H_n exposes the internal chaining state. An attacker can use it to calculate : $\text{Hash}(m \parallel \text{padding} \parallel x)$.

- HMAC: Uses nested keyed hashing so the padded HMAC cannot be directly used as an internal state for extending the message.

- SHA-3:- Uses a sponge construction instead of Merkle-Damgard, where the digest does

Question - 23:-

SHA-256 processes each 512-bit message block through 64 rounds :-

Labelled block diagram:-

512-bit message block

↓
split into 16 words

$W_0, W_1, \dots, W_{15}$

↓
message schedule

Expand to $W_0, W_1, \dots, W_{63}$

↓
Initial hash values,

$H_0, H_1, H_2, H_3, \dots, H_7$

↓
8 working variables,

$a, b, \dots, g, h$

↓
64 compression rounds

↓
Round Function

↓
updated $a, b, c, \dots, h$

↓
Add previous hash

↓
256-bit output hash.

Main stages:

1. message schedule; The 512-bit block is

divided into sixteen-32 bit words:

$W_0, W_1, \dots, W_{15}$.

2. Working Variables:- Eight 32-bit variables are initialized from the current hash state,

$[a, b, c, d, \dots, h]$

3. Round Function:- Each of the 64 rounds uses the current working variables, message word W_j and round constant K_j to produce updated values.

$(a, b, c, \dots, h) \rightarrow (a', b', \dots, h')$.

4. Hash update:- After 64 rounds, the resulting values are added to the previous hash state to produce the new 256-bit intermediate hash.

$[512\text{-bit block} \rightarrow 64 \text{ rounds} \rightarrow 256\text{-bit state}]$

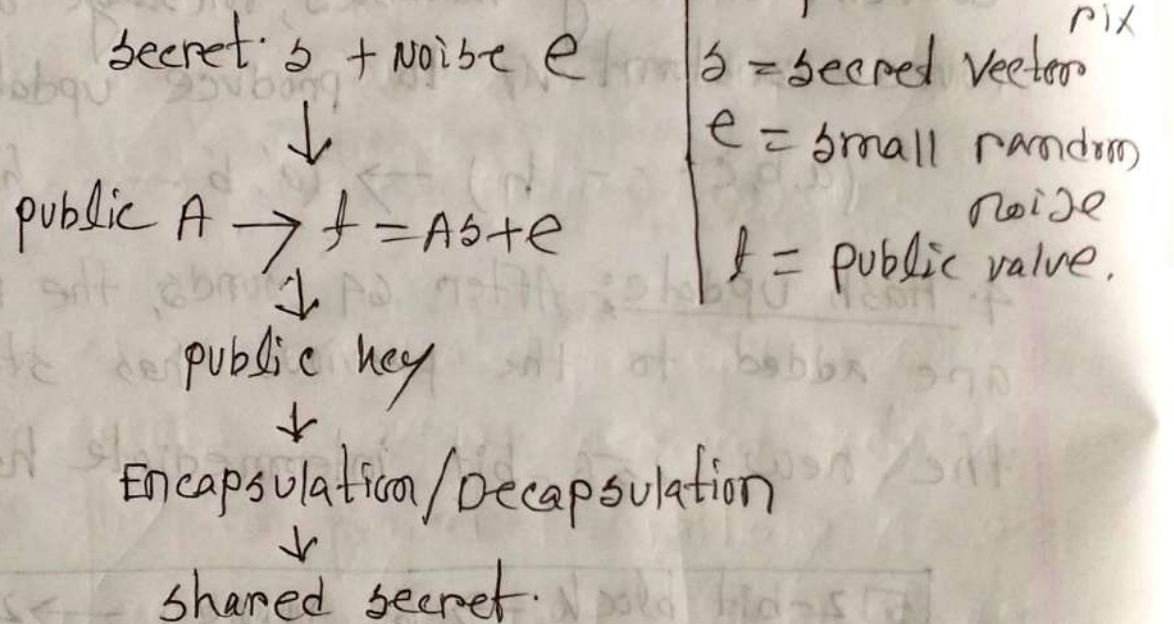
$CA = 1$

$CA = 1$

Question - 35%

⇒ Crystals - kyber is a lattice-based key Encapsulation mechanism (KEM). Its security is based on the hardness of module-LWE (Learning with Errors).

Core idea:- Kyber works with equation of the form $t = As + e$



Why noise is important: without noise,

would be straightforward algebraic problem.
with small random errors,
 $t = As + e$.

the attacker must solve a noisy lattice problem, which is computationally hard.

post-quantum security: Recovering the secret from module-LWE instances is believed to be hard for both classical and quantum computers. Unlike RSA and ECC, no known quantum algorithm such as Shor's algorithm efficiently solves module LWE.

module-LWE hardness $\rightarrow$ post quantum security

Question-36:-

$\Rightarrow$ Kerberos uses a trusted third party to authenticate users without repeatedly sending password.

1. As Exchange:

The client sends: ID_C, ID_{TGS} .

The AS returns: $\{K_{C,TGS}\}_{K_C} + TGT$

where $K_{C,TGS}$ is a session key between the client and TGS. The TGS contains information such as the client identity, session key and ticket lifetime and is protected using the TGS's secret key.

2. TGT Exchange :-

The client sends : TGT + Authentication

The TGS verifies them and returns ;

$\{ K_{c,s} \} K_{c,TGS} + \text{Tickets}$.

3. Client server Exchange :-

The client sends : Tickets + Authenticator

to the service. After successful verification, the service may return $\{ \text{Timestamp} + 1 \} K_{c,s}$.

This provides mutual authentication.

Why TGT avoids repeated password transmission :-

⇒ The client's password-derived key K_c is used only to protect the initial AS response. After receiving the TGT, the client uses the TGT and session key $K_{c,TGS}$ to obtain service tickets.

password → AS only

TGT + session keys → subsequent authentication

2. TGS Exchange :-

The client sends : TGT + Authentication

The TGS verifies them and returns ;

$\{ K_{c,s} \} K_{c,TGS} + \text{Tickets}$

3. Client server Exchange :-

The client sends : Tickets + Authenticator

to the service. After successful verification, the service may return $\{ \text{Timestamp} + 1 \} K_{c,s}$

This provides mutual authentication.

Why TGT avoids repeated password transmission :-

⇒ The client's password-derived key K_c is used only to protect the initial AS response.

After receiving the TGT, the client uses the TGT and session key $K_{c,TGS}$ to obtain service tickets.

password → AS only

TGT + session keys → subsequent authentication

Question - 37 :-

⇒ A plain hash MAC is : $MAC = H(k || m)$.

where, k is secret key and m is the message

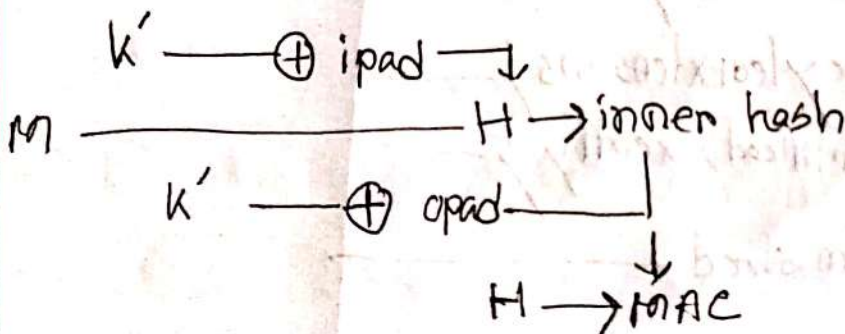
why $H(k || m)$ is vulnerable :-

many hash functions such as SHA-256 use the Merkle-Damgård construction. After computing $H(k || m)$ the hash output effectively represent the internal state after processing $k || m$.

If an attacker knows, m , $H(k || m)$ they can continue the hash computation with an 'extension m' ' and obtain $H(k || m || \text{padding} || m')$ without knowing k . Thus the attacker can create a valid MAC for an extended message.

HMAC construction :-

$$HMAC(k, m) = H((k' \oplus opad) || H(k' \oplus ipad || m))$$



the attacker sees only the final outer hash. They cannot continue the inner hash because its internal state is not directly exposed as the MAC.

simplified numerical example:-

suppos a toy has 15: $H(K) = (\sum x_i) \bmod 100$
Let, $k = 12$, $m = (20, 30)$

plain MAC: $H(k || m) = (12 + 20 + 30) \bmod 100 = 62$

An attacker appends $m' = 40$.

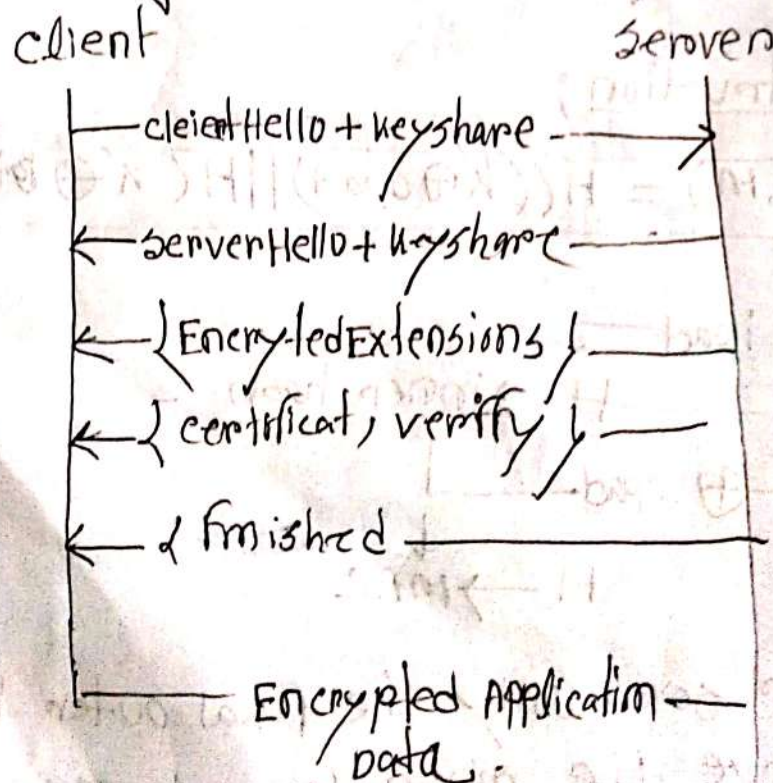
$$H(k || m || m') = (12 + 20 + 30 + 40) \bmod 100 = 2$$

plain $H(k || m)$: vulnerable

HMAC: resistant to length-extension attacks

question-38:-

Labelled Diagram



After server Hello, handshake message are encrypted.

HKDF Key Derivation: Both sides calculate the same ephemeral Diffie-Hellman value

$$Z = (EC) DHE \text{ shared secret}$$

TLS 1.3 uses HKDF - Extract and HKDF - Expand to derive the traffic secrets.

simplified sequence,

(EC) DHE secret



Handshake secret

HKDF - Expand.

Client Handshake Traffic secret

Server " " "

TLS 1.3 vs TLS 1.2

Features

1. Typical full handshake

2. key exchange

3. Forward secrecy only with DHE

4. Handshake encryption starts later

2 RTT

RSA

1 RTT

(EC) DHE

provided by ephemeral.

after server Hello

After server Hello, handshake message are encrypted.

HKDF Key Derivation: Both sides calculate the same ephemeral Diffie-Hellman value

$Z = (\text{EC}) \text{ DHE shared secret}$

TLS 1.3 uses HKDF - Extract and HKDF - Expand to derive the traffic secrets.

simplified sequence,

$(\text{EC}) \text{ DHE shared secret}$



Handshake secret

HKDF - Expand.

Client Handshake Traffic Secret

Server " " "

TLS 1.3 vs TLS 1.2

Features

→ TLS 1.2

TLS 1.3

1. Typical full handshake

2 RTT

1 RTT

2. Key exchange

RSA

(EC) DHE

3. Forward secrecy only with DHE

provided by ephemeral.

4. Handshake encryption starts later

after server Hello

Question - 40 :- A

⇒ Ascon is a lightweight authentication-encryption algorithm standardized by NIST for constrained devices. Compared with AES-128.

Feature

	<u>Ascon</u>	<u>AES-128</u>
Memory footprint	Generally small	Larger implementation.
Energy consumption	Design for low power.	can require more resource on some microcontroller.
Authentication	Built-in-AEAD (encryption + auth.).	AES needs a separate mod/infra
side channel resistance	Requires protected implementation.	Also require protected implementation.

For a battery powered sensor with < 10 KB RAM, Ascon is generally a suitable choice because it was specially designed for constrained environment and provides authenticated encryption in one primitive.